

AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY



Department of Computer Science and Engineering

Course No: CSE 2104

Course Title: Data Structures Lab

Assignment No: 01

Date of Submission: 28/06/26

Submitted by,

Name: MD. Sadman Zuhayer

Student ID: 00724205101048

Lab Section: A2

Academic Semester: Spring 2025

Program: BSc in CSE

Question:

A bookstore maintains a sorted list of unique book IDs to quickly identify whether a book is available in stock. You are required to write a C++ program that uses Binary Search to determine if a specific book ID is present in the list. Additionally, the store often needs to find the first book ID that is not smaller than a given value (lower bound) and the first book ID that is greater than a given value (upper bound) in order to manage restocking and categorization. Given a sorted array of book IDs and a query book ID, your program should output whether the book exists in the list, the lower bound position and the upper bound position corresponding to the query.

Solution:

To solve the question, the Algorithms used are

- 1.Binary Search: To find the exact position of the target in the sorted list
- 2.Lower Bound Search: To find the first position in the sorted list which is greater or equal to the target
- 3.Upper Bound Search: To find the first position in the sorted list which is strictly greater then the target

C++ Code:

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int BinarySearch(vector < int > & book_ids, int x) {
```

```
    int low = 0;
```

```
    int high = book_ids.size() - 1;
```

```
    int mid;
```

```

while (low <= high) {
    mid = (low + high) / 2;

    if (book_ids[mid] == x) {
        return 1;
    }
    else if (book_ids[mid] < x) {
        low = mid + 1;
    }
    else {
        high = mid - 1;
    }
}
return 0;

```

```

}
int UpperBoundSearch(vector < int > & book_ids, int x) {

```

```

    int low = 0;
    int high = book_ids.size() - 1;

    int mid, target = book_ids.size();;

```

```

while (low <= high) {
    mid = (low + high) / 2;

    if (book_ids[mid] > x) {
        target = mid;
        high = mid - 1;
    }
    else {
        low = mid + 1;
    }
}

```

```

    }
    return target;
}

int LowerBoundSearch(vector < int > & book_ids, int x) {

    int low = 0;
    int high = book_ids.size() - 1;

    int mid, target = book_ids.size();

    while (low <= high) {
        mid = (low + high) / 2;

        if (book_ids[mid] >= x) {
            target = mid;
            high = mid - 1;
        }
        else {
            low = mid + 1;
        }
    }
    return target;
}

int main() {

    int n, x, flag = 0;
    vector < int > book_ids;

    cin >> n;
    for (int i = 0; i < n; i++) {

        cin >> x;

```

```

        book_ids.push_back(x);
    }

    cin >> x;

    if (BinarySearch(book_ids, x)) {
        cout << "Book found" << endl;

        cout << "Lower Bound index:" << LowerBoundSearch(book_ids, x) << endl;

        cout << "Upper Bound index:" << UpperBoundSearch(book_ids, x) << endl;

    } else {
        cout << "Book not found" << endl;
    }

}

```

Sample Input

7	Book Found
10 20 30 40 50 60 70	Lower Bound Index: 3
40	Upper Bound Index: 4

Sample Output